



E-CELL

AI & AUTOMATION

DOMAIN

T A S K 1



PROBLEM STATEMENT

Financial reports contain a wealth of critical information, but reading and analyzing them manually is slow, error-prone, and inefficient. Your task is to build an end-to-end document intelligence and classification system using a publicly available dataset of 10-K SEC filings.

The system must take raw financial filing documents, preprocess them, extract meaningful text sections, convert text into machine-readable features, train and compare classification models, and finally deploy the best model through a clean API endpoint.

DATASET

Link : https://huggingface.co/datasets/winterForestStump/10-K_sec_filings

10-K SEC filings are annual financial reports submitted by publicly traded companies to the U.S. Securities and Exchange Commission. They contain rich, structured sections including risk factors, business overviews, management discussions, and audited financial statements, making them ideal for NLP classification tasks.

Classification Setup

Using the dataset, define a classification problem. Suitable targets include:

- Financial risk classification (high / medium / low risk)
- Sentiment-based classification (positive / negative / neutral outlook)
- Any other well-defined label structure available in or derivable from the dataset.

⚠ IMPORTANT NOTE: The classification target must be clearly defined and justified in your Model Report. Ambiguous or undefined label setups will negatively impact your evaluation score.

What You Need to Build

Your project must implement a **five-stage pipeline**. Each stage builds on the previous. Below are the first three stages in detail.

Stage 1: Data Extraction & Preprocessing

- Load the 10-K SEC filings dataset from Hugging Face.
- Extract the raw text from filings.
- Clean the text by removing noise, extra symbols, boilerplate, and irrelevant content.
- Split or identify important sections where possible:
 - Risk Factors
 - Business Overview
 - Management Discussion & Analysis (MD&A)
 - Financial Statements

Stage 2: Feature Engineering

Convert cleaned text into machine-readable numeric features using one or more of the following:

TF-IDF Vectorization

01

- › Term frequency-inverse document frequency
- › Sparse matrix representation
- › Fast, interpretable baseline

Word Embeddings

02

- › Word2Vec, GloVe, FastText
- › Dense vector representations
- › Captures semantic similarity

Sentence Embeddings

03

- › Sentence-BERT, Universal Sentence Encoder
- › Contextual, document-level vectors
- › State-of-the-art NLP representations

Custom Features (Optional)

04

- › Document length, section counts
- › Keyword frequencies, readability scores
- › Any engineered signal you find useful

Stage 3: Classification Model Training

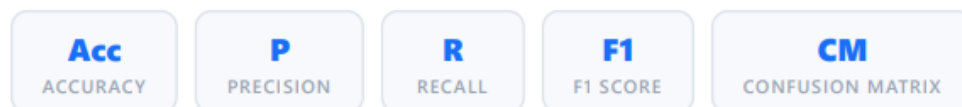
Train and compare all three of the following boosting models on your feature set:

MODEL	LIBRARY	KEY CHARACTERISTICS
XGBOOST	xgboost	Gradient boosting with regularization; widely used in competitive ML
ADABOOST	sklearn	Adaptive boosting; focuses on misclassified samples iteratively
CATBOOST	catboost	Handles categorical features natively; robust to overfitting

Stage 4: Model Evaluation

Evaluate all three trained models and produce a clear performance comparison report.

Required Metrics for Each Model:



Clearly state which model performs best and provide a reasoned justification, not just raw numbers. Discuss class-level performance, trade-offs, and any patterns in misclassifications.

Stage 5: API Deployment

- Select the best-performing model from your evaluation.
- Serialize and save the trained model (e.g., .pkl, .joblib, or .cbm).
- Deploy a FastAPI prediction endpoint.
- The endpoint must accept raw input text and return a structured prediction result (label + confidence).
- Swagger / OpenAPI docs should be auto-generated and accessible at /docs.

DEPLOYMENT ENDPOINT:

POST /predict • Input: { "text": "..." } • Output: { "label": "...", "confidence": 0.xx }

Minimum Requirements

- Data loading and preprocessing implemented
- Text cleaning pipeline (noise removal, normalization)
- Section extraction or document segmentation
- Feature engineering (at minimum TF-IDF)
- Training of XGBoost, AdaBoost, and CatBoost
- Performance comparison across all three models
- Confusion matrix and full metric reporting
- Model saving / serialization
- FastAPI application code
- Working /predict endpoint with correct input/output

Deliverables

1 • Source Code

A well-organized GitHub repository with clean, modular, readable Python code. Each stage of the pipeline should be in a separate file. Avoid monolithic notebooks for final submission.

2 • Model Report

A concise written report (PDF or Markdown) covering:

- Preprocessing decisions and text cleaning steps taken
- Features used and why they were chosen
- Model comparison, metrics table and interpretation
- Best model selection with justification

3 • Live Demo

Be ready to demonstrate the following during evaluation:

- Full preprocessing pipeline running on sample data
- Model training output and logs
- Evaluation results and confusion matrices
- FastAPI /predict endpoint responding to live queries

Suggested Folder Structure

```
project/
├─ data/           # raw and processed datasets
├─ notebooks/      # exploratory analysis
├─ src/
│   ├─ preprocess.py # text cleaning & section extraction
│   ├─ features.py   # TF-IDF / embeddings
│   ├─ train.py      # model training loop
│   ├─ evaluate.py   # metrics & confusion matrix
│   └─ utils.py      # helpers & logging
├─ api/
│   └─ app.py        # FastAPI application
├─ models/         # saved model files
├─ README.md
└─ requirements.txt
```



SUBMISSION GUIDELINES

Meet the deadlines, late submissions will not be entertained PDF format of the ppt must be submitted.

For any clarifications reach out to:

Buvanesh- +91 72006 27711

Aarya - +91 84689 48933